ANT.UI

User Manual & Reference Guide

Created by: Andrés Martínez (andres.martinez@ua.es)
Reviewed by: Carlos Sabater (carlos.sabater@ua.es)

Version: 1.1
Date: November 2026

Brief code description

ANT.UI is a graphical interface designed to streamline the construction of molecular junction inputs for ANT.Gaussian (ANT). The application allows users to build a complete junction geometry from scratch in real time, manipulating electrodes and molecules interactively in three dimensions, and to export ready-to-run input files for ANT and Gaussian with a single click. Advanced sequential-output tools further extend these capabilities, enabling automated electrode-pulling sequences, surface scans, and step-wise rotation studies. Together, these features substantially reduce the time required to set up transport calculations, both for experienced ANT users and for newcomers to theoretical molecular electronics.

Contents

1 Installation and configuration **3**

2 The user interface **5**

3 General workflow **8**

4 Basis sets **9**

5 Spin-orbit coupling **10**

6 Optimization **10**

7 DFT+U **13**

8 Advanced functionalities **14**

8.1 Pull Assistant 14

8.2 Grid Assistant 15

8.3 Rotation Assistant 16

8.4 Examples of use 17

9 Worked examples **18**

10 Multiple molecules in one input **20**

1 Installation and configuration

ANT.UI is a Python application and can be run directly from its source files. Python 3.8 or later is required, along with the packages listed in `requirements.txt`. Install them with:

```
pip install -r requirements.txt
```

On Linux, `tkinter` is not bundled with the default Python installation and must be installed separately via the system package manager (e.g. `sudo apt install python3-tk` on Debian/Ubuntu). On Windows and macOS it is included with the standard Python installer. To launch the application, run:

```
python ANT.UI.py
```

The program can be started from any folder: it locates its data files relative to its own location. On Windows, double-clicking `ANT.UI.bat` starts it without a console window; to get a desktop icon, right-click the file and choose *Send to → Desktop (create shortcut)*.

Alternatively, a standalone executable can be compiled with PyInstaller using the command at the top of `ANT.UI.py`. It can also be acquired from the GitHub repository as a precompiled version.

Testing the installation

ANT.UI ships with an automated test suite that checks the generated inputs without Gaussian or a cluster. Every atom must have a basis set, plus an ECP when its basis requires one; charge and multiplicity must agree with the electron count; the helper scripts must be valid Python; and every sequence must be complete. For optimisation chains, the tests write the Gaussian output that an unchanged optimisation would leave, run the generated helper script, and check that the geometries it propagates are the requested ones. From the program folder, run:

```
pip install pytest
python -m pytest
```

Configuration file

Before using ANT.UI for the first time, edit the `USER_CONF.txt` file to match your cluster environment. The file is parsed by searching for the delimiter `#%%`, reading the keyword on the following line, and treating all subsequent lines up to the next `#%%` as the value for that keyword. Any text outside a keyword block is ignored, so the file can be freely annotated or used to store multiple cluster configurations. Inside a block, `#` starts a comment that is not copied to the generated files, except in **JOBHEAD**, which is copied verbatim so that scheduler

directives such as `#SBATCH` are preserved.

The **Site config** button in the main window shows the configuration exactly as ANT.UI has read it, flags common problems (for example, a job header that is still the shipped placeholder), and can reload the file after you edit it, without restarting the program.

The supported keywords are:

- **ANTDIRcall**: Path to the ANT installation directory on the cluster, containing the modified `L502` and `L101` Gaussian link files. All new users must set this.
- **JOBHEAD**: Header lines written at the top of the generated `launchANT.sh` submission script. Tailor this to your cluster's scheduler (e.g. PBS or Slurm) and set the number of cores here. The shipped value is a placeholder, marked `PLACEHOLDER`, and every export summary warns until it is replaced.
- **GAUSSIANHEAD**: Opening lines of the Gaussian input file (memory, processor count, etc.). The number of processors should match the value set in **JOBHEAD**.
- **G09call**: The command used to invoke Gaussian on your cluster (e.g. `g09`). If your cluster calls Gaussian with a different command, set it here.
- **PYTHONcall**: The command used to invoke Python 3 on the cluster (e.g. `python3`). ANT.UI embeds Python helper scripts in sequential outputs that are launched via this call.
- **ANTpredet**: Fixed lines written to the `.ant` input file for every job. These correspond to the standard ANT convergence and accuracy parameters. Additional keywords may be appended automatically by ANT.UI depending on the calculation type.
- **ZMAX**: Upper limit in Ångström of the z-position slider for the top electrode. Defaults to 20.0 Å. Increase this value if your junction geometry requires a larger electrode separation.
- **U_ELECTRON_SCREENING**: Multiplicative factor applied to the DFT+U Hubbard parameter before it is written to the `.ant` file. Defaults to 1.0. A value of 0.7 is typical and accounts for the screening of the bare molecular U by the metallic electrodes. See Section 7 for details.
- **OUTPUTDIR**: Folder in which new calculations are created (optional; defaults to `Outputs`). It can also be changed for the current session from the main window.

2 The user interface

The purpose of ANT.UI is to construct the three-dimensional geometry of a molecular junction interactively. The junction consists of three components, a bottom electrode, a molecule, and a top electrode, whose relative positions fully define the input geometry. ANT.UI allows the user to move the top electrode and the molecule independently; the bottom electrode acts as the fixed reference frame.

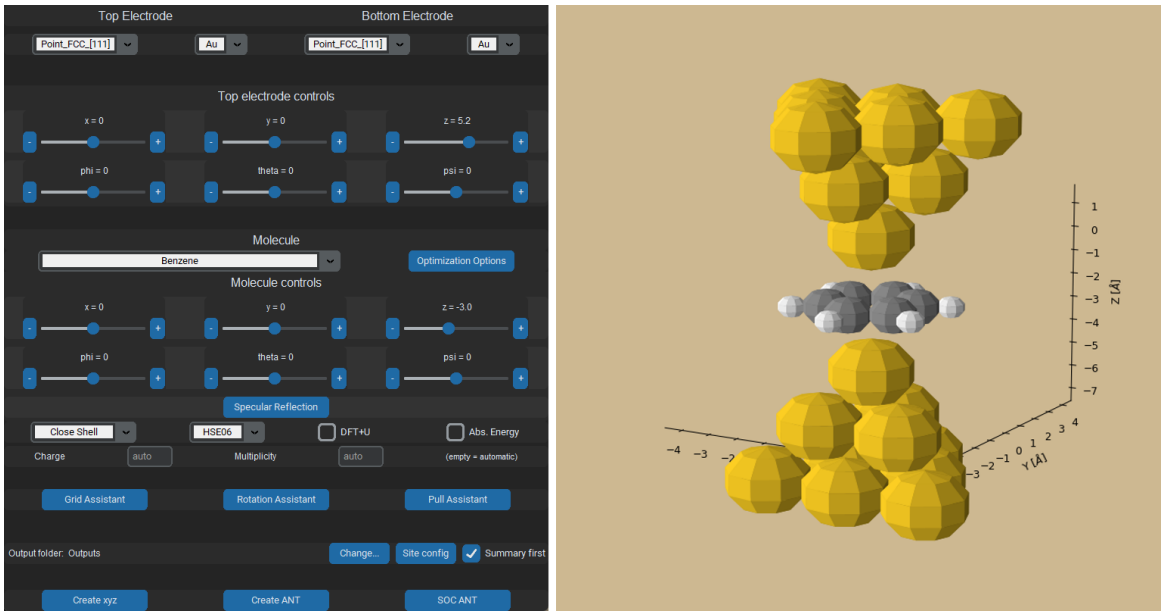


Figure 1: The ANT.UI interface, showing the controller window (left) and the 3D viewer (right).

As shown in Fig. 1, the interface consists of two windows: the **controller** and the **viewer**. The viewer renders an interactive 3D representation of the junction. The outermost layer of each electrode is hidden in the viewer because it serves as a Bethe-lattice anchor layer in ANT and is treated differently from the rest of the structure (see Fig. 4 for how this layer, and the one adjacent to it, are handled). The controller provides all controls for modifying the geometry, selecting components, and configuring DFT settings.

The controller is organized from top to bottom as follows.

Electrode configuration

The first row allows the user to select the electrode type and element independently for the top and bottom electrodes. Electrode geometries are stored as XYZ files in the `ElectrodesDFT` folder; users may add custom geometries to this folder at any time. The shipped templates are face-centred cubic (FCC) tips (`Point`) and slabs (`Block`) with (111) and (100) orientations. All

electrode files in the folder must use gold atoms as a template, since other elements are obtained by rescaling the gold geometry by the ratio of metallic radii, which for FCC metals equals the ratio of lattice parameters. This rescaling only changes the geometry; the basis sets, pseudopotentials and Bethe-lattice parameters of each element are assigned separately (Sections 4 and 5).

All files must represent a *top* electrode; ANT.UI generates the bottom electrode from the same file. For (111) templates the bottom electrode is the top one rotated by 180° about y and 60° about z, so the ABC stacking of the FCC lattice continues across the junction as in a single crystal; other templates are mirrored through the xy plane. The same transformation is applied whether the electrode is the start-up default or is selected later.

Top electrode controls

The next section contains the position and orientation controls for the top electrode. The upper row of sliders translates the electrode along the x, y, and z axes; the lower row rotates it around each axis. The orientation depends only on the three angles shown (applied about x, then y, then z), not on the order in which the sliders were moved. Note that ANT requires the Bethe lattice to be aligned along the z axis, so rotating the electrode about the x or y axis is generally not recommended.

Molecule selection

A pre-loaded catalogue of molecules is available from the **Molecule** drop-down menu. To add a custom molecule, place an XYZ-format text file in the `Molecules` subfolder; the new entry will appear in the menu after restarting the application. The XYZ format expected by ANT.UI is:

```
12
LSDA 3.5 | PBE 4.0
C  0.000  0.000  0.000
H  1.000  0.000  0.000
...
```

The first line is the atom count. The second line is the comment line and may optionally contain DFT+U values for one or more functionals (see Section 7); leave it blank if DFT+U is not needed. Each subsequent line gives an element symbol followed by x, y, z coordinates in Ångström. To generate a purely metallic junction without a molecule, select the `PureMetallic` entry.

The **Optimization Options** button, located on the same row as the molecule selector, opens the optimization configuration window described in Section 6.

Molecule controls

The molecule controller mirrors the top-electrode controller. An additional **Specular Reflection** button is provided for chiral molecules: pressing it applies a mirror reflection to all atomic coordinates, toggling the handedness of the molecule without requiring a second file.

DFT settings

This row sets the electronic-structure parameters for the output files: the spin treatment (closed shell or open shell), the exchange-correlation functional (chosen from a list of common options), and optionally DFT+U (see Section 7). The **Abs. Energy** checkbox, when enabled, instructs ANT.UI to include an additional Python script in the output that generates the necessary files to calculate the absorption energy of the molecule for the configuration. This is useful for constructing potential-energy absorption curves alongside transport data, for example during an electrode-pulling sequence.

The **Charge** and **Multiplicity** fields below are left empty for automatic values. ANT.UI counts the electrons of the junction after removing the cores replaced by each pseudopotential (which depend on the basis assigned to each atom, Section 4). An even count gives a neutral singlet. An odd count gives charge +1 and a singlet for closed-shell calculations, or a neutral doublet for open-shell ones. Typing an integer in either field overrides the automatic value; the export summary warns if the resulting combination is impossible for the electron count.

Advanced functionalities

The three buttons in this row, **Pull Assistant**, **Grid Assistant**, and **Rotation Assistant**, open pop-up windows for generating sequential outputs. These are described in detail in Section 8. Read that section before using them.

Output creation

The last rows contain the output folder and the output buttons. The **Output folder** row shows where new calculations will be created (`Outputs` by default, or the **OUTPUTDIR** of `USER_CONF.txt`); **Change...** selects another folder for the session. Each export creates a new subfolder named after the molecule plus a four-digit index, one above the highest index already present, so a deleted folder never causes a clash.

- **Create xyz**: writes only the XYZ coordinate file for the junction geometry.
- **Create ANT**: generates all input files required to run a transport calculation with ANT (Gaussian `.gjf`, ANT `.ant`, and a submission script `launchANT.sh`).
- **SOC ANT**: generates the same set of files as **Create ANT** but with spin-orbit coupling enabled, provided the electrode element is supported (see Section 5).

Export summary and checks

With **Summary first** ticked (the default), every export first opens a summary of the complete model; nothing is written until **Generate** is pressed. The summary lists:

- the number of atoms in each part (bottom electrode, molecule, top electrode), the Bethe-region and anchor-layer atoms, and the closest distances between the parts;
- the functional, the electron count, and the charge and multiplicity (automatic or user-defined);
- the basis set and pseudopotential of every element in every region, with the file each one comes from, including the spin-orbit and all-electron cases;
- the Bethe-lattice parameters, spin-orbit factors and DFT+U settings, when used;
- the optimisation constraints of each region and the chain mode;
- the sequence settings (steps, angles or grid) and the site configuration.

Warnings are shown in red, for example overlapping atoms (closer than 1 Å, also checked at the last step of a compression), an impossible charge and multiplicity, a placeholder job header, or a chain mode with nothing to optimise. The same text is saved as `ANTUI_summary.txt` in the output folder, so every calculation keeps a record of how it was generated.

After writing, ANT.UI checks that every generated helper script is valid Python and reports in the status line the folder and the number of configurations created. If anything fails, an error message is shown and the incomplete folder is removed.

3 General workflow

The recommended procedure for building a junction and generating outputs is as follows.

- **Select all components first.** Choose the electrode type and element for both electrodes, and select the molecule, *before* making any positional adjustments. Changing the electrode element after moving it does not reset its position, but it does rescale the geometry, which will shift atoms from their intended locations.
- **Position the molecule, then the top electrode.** The bottom electrode is fixed and serves as the reference. It is easiest to first bring the molecule into contact with the bottom electrode using the molecule sliders, and then lower the top electrode to complete the junction.

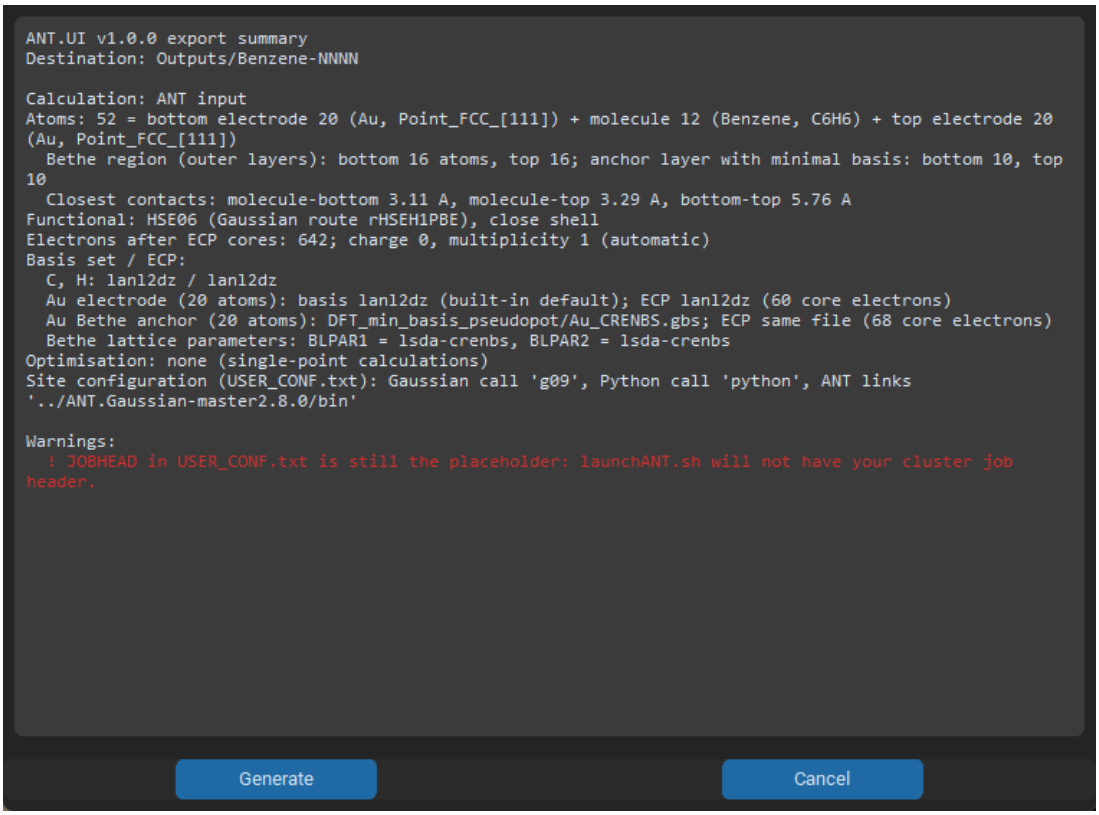


Figure 2: Export summary shown before writing the files of Example 1 (Section 9). Warnings appear in red at the bottom; here, the job header of `USER_CONF.txt` is still the shipped placeholder.

- **Configure DFT settings.** The functional, spin treatment, and optimization options are persistent across the session; they can be set at any point and will apply to all subsequent outputs.
- **Generate outputs.** Once the geometry and settings are finalised, press the appropriate output button and read the export summary (Section 2) before pressing **Generate**. Multiple outputs can be generated in a single session; creating an output does not alter any positional or DFT settings.

4 Basis sets

ANT.UI assigns basis sets to atoms depending on their role in the junction.

Bethe-lattice support atoms (the outermost layer of each electrode) are assigned a minimal basis. ANT.UI looks for a file named `Element_CRENBS.gbs` (e.g. `Au_CRENBS.gbs`) in the

`DFT_min_basis_pseudopot` folder. Each file must follow the Gaussian format: the basis set first, then the pseudopotential, separated by `****`.

Standard electrode atoms (the layers shown in the viewer) are assigned a larger basis. ANT.UI looks for a matching file in `DFT_basis_pseudopot` using the same naming convention. If no file is found for a given element, `lanl2dz` is used as the default.

Molecule atoms always use `lanl2dz`, for any element (with its pseudopotential for elements beyond neon).

Custom basis sets can be added to either folder at any time; the files are loaded at startup. The element is taken from the file name up to the first underscore (`Au_CRENBS.gbs` is used for Au), so a file can be disabled by renaming it, for example with a prefix. The export summary lists, for every element and region, the file its basis and pseudopotential come from, or `lanl2dz (built-in default)`.

5 Spin-orbit coupling

ANT supports spin-orbit coupling (SOC) calculations through a specialised set of basis functions and SOC parameters. The files required for this are stored in the `SOC_params` folder. Only elements with entries in this folder support the **SOC ANT** output button; if either electrode element is unsupported, the button is disabled automatically. The currently supported elements are: Au, Pt, Cu, Ag, and Al.

In a SOC input, the electrode atoms outside the Bethe anchor layer receive the SOC basis of their element (`Element_SOC_basis.gbs`) and, when the element has one, the matching pseudopotential (`Element_SOC_pseudopot.gbs`); the Cu and Al SOC basis sets are all-electron and need none. Each electrode is treated separately, so mixed junctions (for example an Au bottom electrode and a Cu top electrode) receive the correct basis and pseudopotential for each region; the export summary lists them. The spin-orbit factors written to the `.ant` file (`SOCFACATOM`) are applied to the electrode atoms outside the Bethe region.

6 Optimization

Geometry optimisation of the junction can be performed in Gaussian before the transport calculation. The **Optimization Options** window, opened from the main controller, configures how the structure is relaxed.
As shown in Fig. 3, the window is divided into five regions corresponding to the five structural components: the Bethe-lattice support layer and the active region of each electrode, and the molecule. Each region can be assigned one of the following optimisation modes:

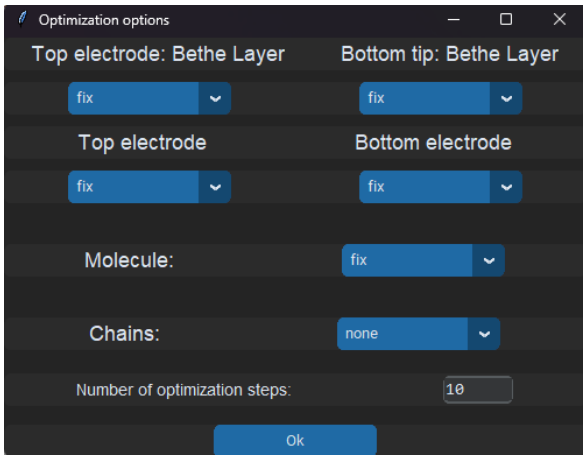


Figure 3: The Optimization Options window.

- **fix**: atoms in this region are frozen.
- **z**: atoms are allowed to relax only along the z axis.
- **all**: atoms are free to relax in all three directions.

The Bethe-lattice support layers are restricted to **fix** or **z** only, since the lattice structure of these layers must be preserved for ANT to attach the semi-infinite electrodes correctly. When **z** is chosen, the support region keeps its bulk structure in two stages. In the Gaussian input, all atoms of a plane share a single z variable, so each plane stays flat and moves rigidly along z. Gaussian 09 cannot tie different planes together, so they relax independently during the optimisation. Afterwards, the connector script keeps the innermost support plane (the one in contact with the active electrode atoms) at its relaxed position and moves the outer planes, including the anchor layer, back to their original distances from it. The geometry passed to ANT, or to the next step of a sequence, therefore has the original interlayer spacing of the support region.

The molecule has an additional option, **all, some z**, which opens a sub-window where the indices of specific atoms can be entered. Those atoms will be constrained to move only along z, while the rest of the molecule remains free. This is useful for preventing the molecule from drifting laterally during optimisation.
The **Chains** option controls how optimisation interacts with the advanced sequential outputs:

- **none**: no optimisation is performed at any step.
- **first step**: only the first geometry in the sequence is optimised in Gaussian; all subsequent geometries are generated from the relaxed first-step structure by applying the displacement, rotation or lateral shift of each step to it. Only the first configuration is relaxed; the others are not re-optimised.

- **every step**: each step in the sequence is initialised from the optimised output of the previous one, forming a chain of relaxations. This is the most computationally intensive option but produces the most physically realistic sequence.

In both chain modes the first folder contains the connector script, and the submission script runs the steps in order.
The **Number of optimization steps** field sets the maximum number of optimisation cycles passed to Gaussian. Molecular junctions are challenging systems for standard geometry optimisers and rarely reach the default convergence thresholds. A value between 10 and 30 steps is usually sufficient for most practical junctions.

When any optimisation mode is active, ANT.UI includes a Python connector script in the output folder. This script reads the Gaussian output and constructs the ANT input from the relaxed geometry. To disable ANT in the pipeline and run only the Gaussian optimisation, remove the `L502` and `L101` substitution lines from both the connector script and the Gaussian input file.

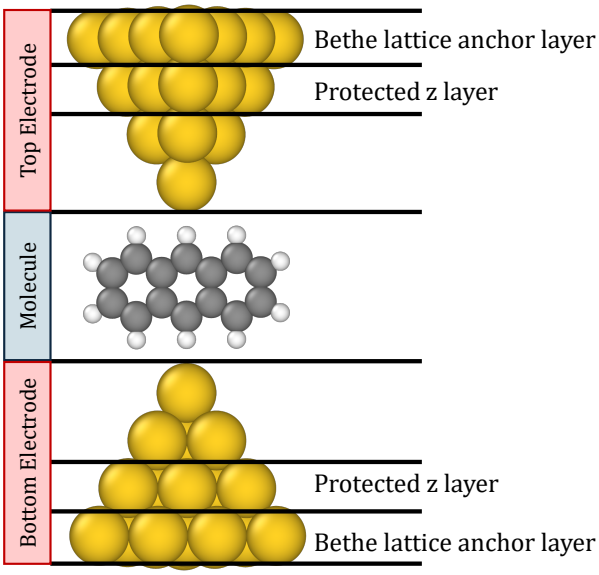


Figure 4: Layer structure of an electrode as handled internally by ANT.UI, shown here for a top electrode with the molecule below. The **Bethe-lattice anchor layer** is always written to the output as the semi-infinite Bethe-lattice attachment with a minimum basis-set, regardless of the optimisation settings, and is hidden in the 3D viewer (Fig. 1). The adjacent **protected z layer** belongs, together with the anchor layer, to the Bethe-lattice support region of Section 6. It may be frozen or, if set to **z**, relaxed along the transport axis as a flat plane, but never with **all**. After the optimisation the anchor layer is put back at its original distance from it, so that the Bethe lattice attaches to an undistorted block.

7 DFT+U

DFT+U is an ANT feature that corrects the Kohn-Sham gap of a molecule by adding a Hubbard-U parameter to the GGA Hamiltonian. The appropriate value of U can be estimated from isolated-molecule Gaussian calculations using:

$$U_{\mathrm{GGA}} = \left| E^{\mathrm{KS}}_{\mathrm{HOMO}} - \left(E[n+1] - E[n]\right) \right| + \left| E^{\mathrm{KS}}_{\mathrm{LUMO}} - \left(E[n] - E[n-1]\right) \right| \tag{1}$$

where $E[n]$ is the total DFT energy of the molecule with n electrons (n being the number of electrons of the neutral species), and $E^{\mathrm{KS}}_{\mathrm{HOMO}}$ and $E^{\mathrm{KS}}_{\mathrm{LUMO}}$ are the frontier orbital energies from the Kohn-Sham calculation.

Once U has been computed for a given molecule and functional, store it in the comment line (second line) of the molecule's XYZ file. For example, to assign $U = 2.86$ eV for the PBE functional, write `PBE 2.86` on that line. Multiple functionals can be listed simultaneously, separated by `|`:

```
PBE 2.86 | BLYP 2.52
```

The DFT+U checkbox in the main controller becomes active only when the selected molecule has a stored U value for the currently chosen functional. When checked, ANT.UI writes the necessary ANT keywords to the `.ant` file. The value written is scaled by the electron screening factor from `USER_CONF.txt` (default 1.0; a value of 0.7 is commonly used to account for metallic screening of the molecular U).

8 Advanced functionalities

The three advanced-functionality buttons open pop-up windows for creating *sequential outputs*: sets of related ANT inputs that together constitute a multi-step calculation. The relationship between steps depends on the chosen assistant and the optimisation chain setting from Section 6. Pressing **Ok** in any of these windows opens the export summary and then generates the full output sequence in the output folder: one numbered subfolder per configuration, a single submission script `launchANT.sh` that runs them in order, a multi-frame XYZ file with every configuration, and a small text file recording the scan (`pull_data.dat`, `grid_data.dat` or `rotation_data.dat`).

8.1 Pull Assistant

The Pull Assistant generates a sequence of outputs in which the electrode separation is varied in uniform steps along the z axis, simulating the stretching or compression of a junction.

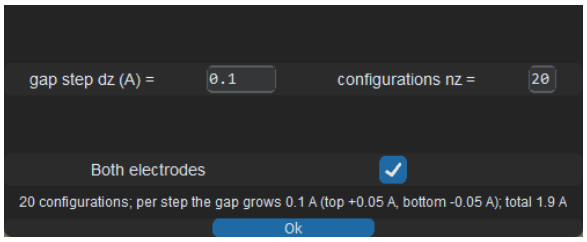


Figure 5: The Pull Assistant window.

The user specifies the number of configurations n_z and the gap step dz (in Ångström). dz is always the change of the electrode–electrode gap per step: configuration k ($k = 0, \ldots, n_z - 1$) has the gap of the starting junction plus $k\,dz$. If **Both electrodes** is unchecked, only the top electrode moves, by $k\,dz$; the bottom electrode and the molecule stay in place. If it is checked, the top electrode moves by $+k\,dz/2$ and the bottom electrode by $-k\,dz/2$, so the junction is stretched symmetrically about the molecule. A positive dz corresponds to stretching (pulling); a negative dz corresponds to compression (pushing). The window shows these displacements as you type, and `pull_data.dat` records them for every configuration. For example, $dz = 0.1$ Å, $n_z = 20$ with both electrodes moving gives 20 configurations in which each electrode moves 0.05 Å per step, for a total gap increase of 1.9 Å.

Each step produces a complete set of ANT input files. When optimisation is enabled, the necessary Python connector scripts and a master submission script are also generated. Note that the master script provides a convenient starting point but may require adaptation to your specific cluster configuration.

The Pull Assistant combined with **every step** optimisation produces a sequence that closely approximates a quasi-static break-junction simulation.

8.2 Grid Assistant

The Grid Assistant generates a two-dimensional grid of outputs by scanning the top electrode over a set of lateral positions at fixed height. This mimics an STM measurement where the transmission is evaluated at each grid point.

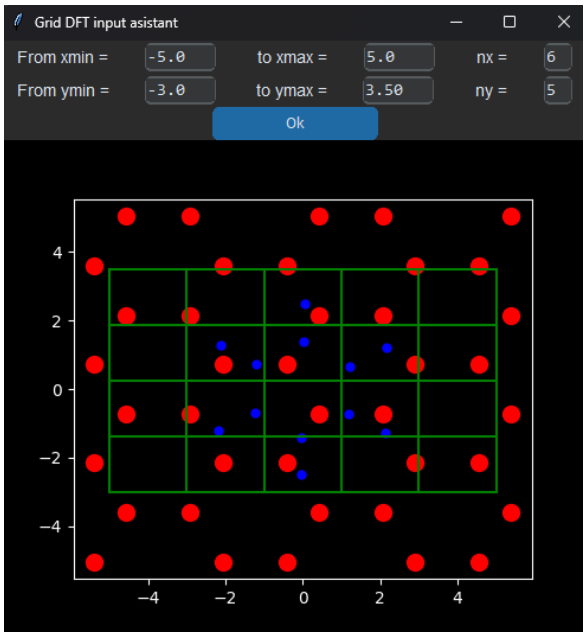


Figure 6: The Grid Assistant window, shown here with a benzene molecule on a gold slab.

The pop-up displays a top-down view of the junction: molecule atoms in blue, bottom-electrode atoms in red, and the user-defined grid in green. The user sets the x and y limits of the scan area and the number of grid points in each direction. The height of the top electrode (z coordinate) is taken from the current session state and held constant across the scan; only the lateral position of the tip varies. The $n_x \times n_y$ points include the limits and are visited in a serpentine order (along y, reversing direction at each new x), so consecutive configurations are always neighbours; `grid_data.dat` lists the tip position of every numbered configuration. The local density of states is requested on the tip apex atom.

Combining the Grid Assistant with **first step** optimisation is a practical strategy: the molecule is first relaxed in place, and the grid scan then uses the relaxed geometry as its starting point.

8.3 Rotation Assistant

The Rotation Assistant generates a sequence of outputs in which a selected part of the junction is rotated about the z axis by a uniform angular increment. This is useful for studying how conductance varies with the relative orientation of a molecule or electrode.

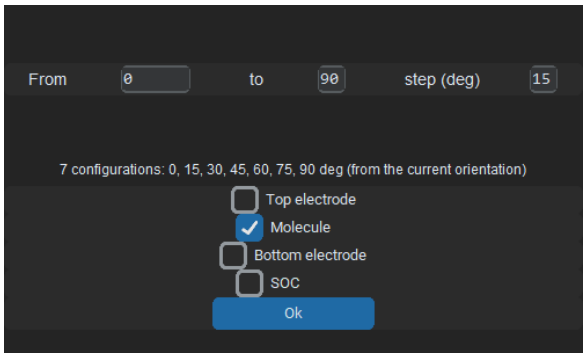


Figure 7: The Rotation Assistant window.

The user selects which components to rotate (top electrode, molecule, and/or bottom electrode), the starting angle (**From**), the final angle (**to**) and the angular increment **step**, all in degrees. Both ends are included, so the number of configurations is $(\text{to} - \text{from})/\text{step} + 1$: for example, from 10 to 70 with a step of 20 gives four configurations, at 10, 30, 50 and 70°. The window lists the angles as you type. The angles are measured from the orientation shown in the viewer (an angle of 0 reproduces the current junction), positive angles turn counter-clockwise when viewed from $+z$, and each selected component turns about the vertical axis through its own centre. After the export, the components are returned to their previous orientation. The **SOC** checkbox generates the sequence with spin-orbit coupling enabled.

Before committing to a full rotation sequence, it is advisable to use the manual rotation sliders in the main controller to preview a few intermediate orientations and verify that the resulting geometries are physically reasonable.

8.4 Examples of use

Figure 8 illustrates several applications of the advanced-output tools using benzene, toluene and glycerol as test molecules. Section 9 gives the complete settings used to generate the inputs of each panel.

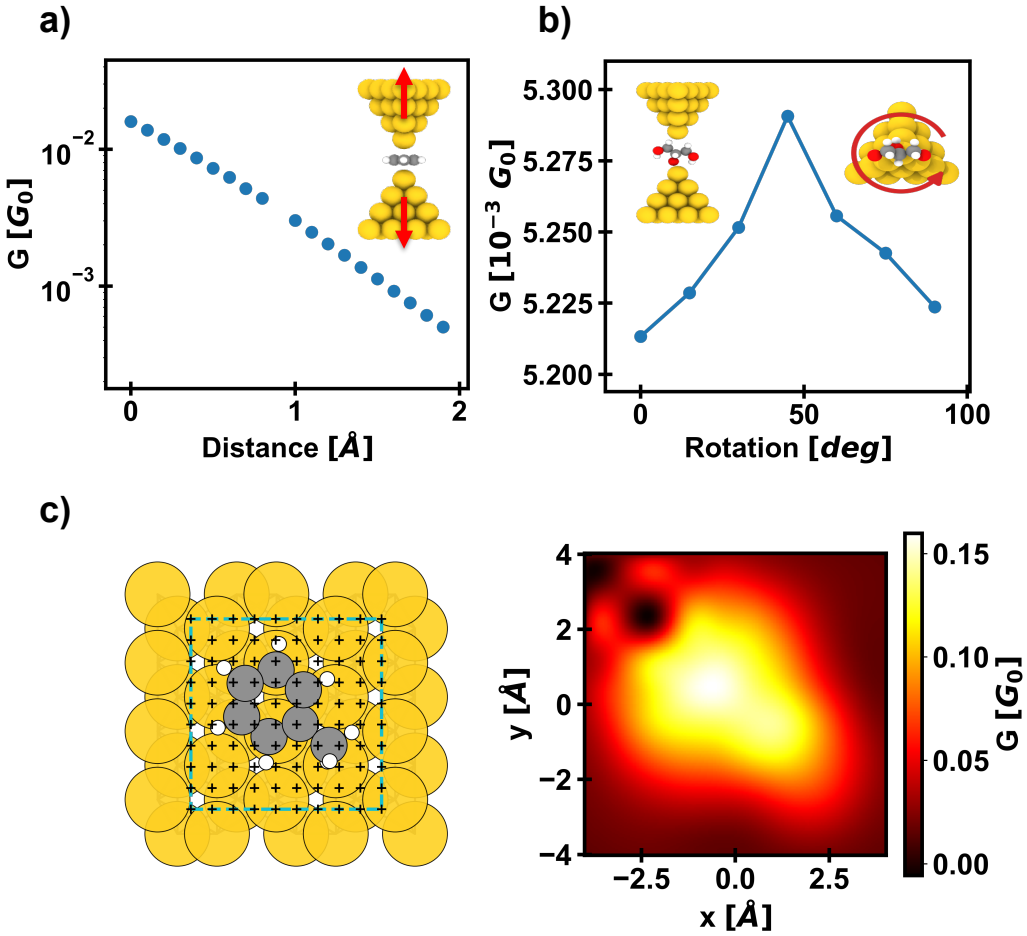


Figure 8: Examples of sequential-output calculations enabled by `ANT.UI`. (a) Transmission vs. electrode separation from a symmetric pulling sequence on benzene, with the molecule relaxed at every step. (b) Transmission vs. rotation angle of a glycerol molecule relative to the electrodes, relaxed at the first step only. (c) Grid scan of a toluene molecule on an Au(111) slab, relaxed at the first step only. Left: top view of the slab and molecule, with a cross marking each scan point. Right: 2D contour map interpolated from the data.

9 Worked examples

This section walks through four complete examples, from an empty session to files ready for the cluster. The first is a single transport calculation; the other three reproduce the inputs of the three panels of Fig. 8. The inputs generated by ANT.UI for all four, together with their export summaries, are included in the `Examples` folder of the repository. Running `python Examples/make_examples.py` regenerates them without the graphical interface. Unless stated otherwise, all sliders are left at 0 and **Summary first** is ticked.

Example 1: a single benzene junction

1. Select `Point_FCC_[111]` and `Au` for both electrodes, and `Benzene` as the molecule.

2. Set the molecule slider $z = -3.0$ and the top-electrode slider $z = 5.2$. The benzene ring now lies flat between the two tip apices.

3. Choose `Close Shell` and `HSE06`; leave **Charge** and **Multiplicity** empty and all optimisation regions on **fix**.

4. Press **Create ANT**. The summary reports 52 atoms (20 + 12 + 20); closest contacts of 3.11 Å (molecule–bottom), 3.29 Å (molecule–top) and 5.76 Å between the apices; 642 electrons after the pseudopotential cores, hence charge 0 and multiplicity 1; `lanl2dz` for C and H, `lanl2dz` for the 20 active Au atoms and `Au_CRENBS.gbs` for the 20 anchor atoms. If **JOBHEAD** has not been edited yet, a warning reminds you.

5. Press **Generate**. The folder `Benzene-0001` contains the Gaussian input `inputBenzene-0001.gjf`, the ANT input `inputBenzene-0001.ant`, the submission script `launchANT.sh`, the geometry `Benzene-0001.xyz` and `ANTUI_summary.txt`.

6. Copy the folder to the cluster and submit `launchANT.sh`. ANT writes the transmission to `T.inputBenzene-0001.dat`.

Example 2: symmetric pull with relaxation at every step (Fig. 8a)

1. Build the junction of Example 1.

2. In **Optimization Options**, set the molecule to **all**, **Chains** to **every step**, and the number of optimisation steps to 20; leave both electrodes and their support layers on **fix**.

3. Open the **Pull Assistant**, enter $dz = 0.1$ and $n_z = 20$, and tick **Both electrodes**. The window reads: 20 configurations; per step the gap grows 0.1 Å (top +0.05 Å, bottom −0.05 Å); total 1.9 Å.

4. Press **Ok**. The summary confirms the chain mode and reports the contacts of the last configuration (3.98 and 4.17 Å). Press **Generate**.

5. The output contains the folders `Benzene-0001_01` to `Benzene-0001_20`, with the ANT input of every step and, for the first step, the Gaussian input with the molecular coordinates as optimisation variables. Each folder has a connector script `link_opt.py`, which writes the input of the next step from the relaxed geometry. `pull_data.dat` records the displacement of each electrode at every step.

6. Submit `launchANT.sh` once: it runs the 20 steps in order, reusing the converged density of each step as the starting point of the next. At the end it collects the transmission files in `T/`, the relaxed geometries in `xyz/` and the optimisation logs in `Tlog/`.

Example 3: rotation of a molecule with first-step relaxation (Fig. 8b)

1. Select `Point_FCC_[111]` and `Au` for both electrodes and `Glycerol` as the molecule. Set the molecule $z = -2.0$ and the top-electrode $z = 6.6$; the molecule is then 2.35 Å from each tip.

2. Choose `PBE`. In **Optimization Options**, set the molecule to **all** and **Chains** to **first step**.

3. Open the **Rotation Assistant**, enter from 0 to 90 with a step of 15, and tick only **Molecule**. The window lists 7 configurations: 0, 15, 30, 45, 60, 75 and 90°.

4. Press **Ok** and **Generate**. Only the first folder, `Glycerol-0001_1`, contains a Gaussian input; on the cluster, its connector script relaxes the molecule once and then writes the inputs of the other six folders by rotating the relaxed molecule about its own vertical axis by 15, 30, . . . , 90°. These later geometries are not re-optimised.

Example 4: tip scan over a molecule on a slab (Fig. 8c)

1. Select `Block_FCC_[111]_40` as the bottom electrode, `Point_FCC_[111]` as the top electrode (both `Au`) and `Toluene` as the molecule. Set the molecule $z = -6.7$ and the top-electrode $z = 3.7$: the toluene lies flat 2.36 Å above the slab, with the tip apex 4.42 Å above the molecule.

2. Choose `PBE`; in **Optimization Options** set the molecule to **all** and **Chains** to **first step**.

3. Open the **Grid Assistant** and set x and y from -4 to 4 with $n_x = n_y = 10$. The green grid in the window covers the molecule with a spacing of 0.89 Å.

4. Press **Ok** and **Generate**. The output has 100 folders (`Toluene-0001_001` to `_100`), and `grid_data.dat` gives the tip position of each. The molecule is relaxed once under the tip at the first point; the other 99 inputs are produced on the cluster by shifting the tip over the relaxed surface.

10 Multiple molecules in one input

ANT treats all non-metallic atoms in the input as a single "molecule", regardless of whether they form one connected fragment or several. This behaviour can be exploited to simulate transport through multi-molecule junctions.

ANT.UI does not provide a dedicated tool for this scenario, but the following manual procedure works reliably. Set up the electrodes and place the first molecule in its desired position, then generate an output. Without moving the electrodes, reposition the molecule to the second desired location and generate a second output. Repeat for each molecule. Finally, open the Gaussian input files from all outputs and copy the atomic coordinates of each molecule into a single input file. When doing so, the atom indices used to identify the top-electrode region in the ANT input must be updated to account for the extra atoms that have been inserted.

Alternatively, a multi-molecule arrangement can be saved as a single XYZ file in the `Molecules` folder. ANT.UI will treat it as one object, and the geometry optimisation tools can then be used to refine the arrangement in the usual way.